THE HARDEST PROBLEM IN SOFTWARE ENGINEERING

Naming Things.

TOM BENNER

Naming Things

The Hardest Problem in Software Engineering

Second Edition

Copyright © 2023 Tom Benner. All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law. For permission requests, write to the publisher, tombenner@gmail.com.

Naming Things

- I. Introduction
  - 1. Purpose of this book
  - 2. Why is naming important?
    - 2.1. Understandable code
    - 2.2. Productivity and happiness
    - 2.3. Career growth
  - 3. Why is naming hard?
    - 3.1. Dynamic, subjective requirements
    - 3.2. Insufficient best practices and tooling
    - 3.3. Short-term costs and long-term value
    - 3.4. How can we make naming easier?
  - 4. Scope of this book
- II. Principles
  - 5. Overview
    - 5.1. List of principles
  - 6. Understandability
    - 6.1. Overview
    - 6.2. Rules
      - 6.2.1. Describe the concept
      - 6.2.2. Use dictionary terms
      - 6.2.3. Use problem-domain terms
      - 6.2.4. Use standard terms for domain-agnostic concepts
      - 6.2.5. Use correct pluralization

6.2.6. Use accurate parts of speech
6.2.7. Include units in measurements
6.2.8. Avoid unconventional single-letter names
6.2.9. Avoid abbreviations
6.2.10. Avoid non-standard symbolic names
6.2.11. Avoid cleverness
6.2.12. Avoid usage of temporary or irrelevant concepts
6.2.13. Consider the audience's familiarity with the name
7. Conciseness
7.1. Overview
7.2. Rules
7.2.1. Use the appropriate level of abstraction
7.2.2. Use words with rich meaning
7.2.3. Omit metadata
7.2.4. Omit implementation details
7.2.5. Omit unnecessary words
8. Consistency
8.1. Overview
8.2. Rules
8.2.1. Obey popular naming conventions
8.2.2. Avoid synonyms
8.2.3. Avoid abbreviations
8.2.4. Use similar names for similar concepts

8.2.5. Use consistent antonyms
9. Distinguishability
9.1. Overview
9.2. Rules
9.2.1. Avoid homographs and near-homographs
9.2.2. Avoid homophones and near-homophones
9.2.3. Avoid polysemes
9.2.4. Avoid names with distinct technical and non-technical meanings
III. Application
10. Overview
11. Tradeoffs
11.1. Overview
11.2. Consistency vs. other principles
11.3. Understandability vs. Conciseness
11.4. Understandability vs. Distinguishability
11.5. Conciseness vs. Distinguishability
12. Identifier types
12.1. Overview
12.2. Classes
12.3. Variables
12.3.1. Booleans
12.3.2. Collections
12.3.3. Hash maps
12.4. Methods
12.5. Method arguments

12.6. Interfaces
12.7. Constants
12.8. Packages/modules/namespaces
13. Style guides
14. Controlled vocabularies
14.1. Further reading
15. Renaming
15.1. Scope
15.2. Principal and interest
15.2.1. Interest costs
15.2.2. Principal costs
15.3. Process
16. Domain-specific names
16.1. Consult domain-related resources
16.2. Consult domain experts
16.3. Consult team members
17. Developing naming skills
17.1. Improving your naming skills
17.1.1. Initial steps
17.1.2. Ongoing learning
17.2. Improving your team’s naming skills
IV. Appendix
18. Common antonyms
19. Visually similar characters
20. Rejected principles
20.1. Length
20.2. Searchability
20.3. Pronounceability

20.4. Meaningfulness
20.5. Austerity
20.6. Accuracy
20.7. Precision
20.8. Concision
References

Acknowledgements

This book is the systemization of the knowledge of many people with experience in software engineering industry and academia. I greatly appreciate the sharing of this knowledge through papers, books, articles, talks, conversations, and other means. That shared knowledge was both the inspiration for and the foundation of this book.

I also want to give a special thank you to everyone who provided insightful feedback that shaped and refined the content of this book: Albert Chae, Andy Wang, David Gage, Joel Eaton, Marian Hlavac, Nathan Miles, Richard Dyce, Rob Nugen, Tiago Boldt Sousa, Tom Clark, and Zeger Knops.

I. Introduction

1. Purpose of this book

The naming of identifiers (classes, variables, functions, etc) is one of the most frequently-used, timeless, and impactful skills in software engineering. However, it’s rarely analyzed, poorly understood, and poorly executed.

This book’s goal is to make software engineering more efficient and enjoyable through better naming. This may sound audacious, but we can achieve it by applying the same practical, rigorous structure that we’ve used to improve other aspects of software engineering.

2. Why is naming important?

> *Your code is for a human first and a computer second. Humans need good names.*
>
> *— Martin Fowler, Refactoring: Improving the Design of Existing Code*

Throughout the modern history of software engineering, many of the field's experts have emphasized naming's importance, and many of software engineering's most well-regarded books on best practices include chapters about naming, including:

- **Clean Code** (Robert C. Martin) — "Variable Names" [1]
- **Code Complete** (Steve McConnell) — "Meaningful Names" [2]
- **The Pragmatic Programmer** (Andrew Hunt, David Thomas) — "Naming Things" [3]
- **The Practice of Programming** (Brian W. Kernighan, Rob Pike) — "Names" [4]
- **Refactoring** (Martin Fowler) — "Mysterious Name", "Rename Variable", "Rename Field" [5]

Software engineers agree. A study found that students and professional software engineers found naming

conventions and identifier naming to be “essential” and “important”, and that the professional engineers “pay more attention to the name of the identifiers than source code comments”. [6] Identifier names are “implicit documentation and are instrumental for program comprehension”. [7]

2.1. Understandable code

Good naming makes code understandable, which in turn makes maintenance (including both enhancements and corrections) more efficient.

Good naming → Understandable code → Efficient maintenance

Codebases are knowledge bases. A codebase stores the knowledge of how an application should behave, and if the codebase is easy to understand, then software engineers can absorb that knowledge efficiently and modify the application easily, quickly, and confidently.

Can we quantify the importance of a codebase’s understandability? Studies show that maintenance consumes 60%-80% of software development life cycle costs. [8] Approximately 40%-60% of this effort is spent understanding the software that is being modified. [9] Thus, 25%-50% of life cycle time (the product of the previous numbers) is spent understanding the software.

Identifiers (names of classes, variables, functions, etc) account for roughly 70% of a codebase [10], so if these identifiers are easy to understand, then the codebase should have good understandability. Studies about the influence of identifier names on comprehension, recall, and defect resolution time reflect this.

- *Comprehension* - The ability of a software engineer to understand the logic, structure, and purpose of the code.
- *Recall* - The ability of a software engineer to remember the logic, structure, and purpose of the code.
- *Defect resolution time* - The duration between when a software engineer starts working on a defect (a bug) and when they resolve the defect.

Before modifying code, a software engineer must comprehend it. After the initial comprehension, recall enables engineers to make modifications without needing to relearn the code's logic, structure, and purpose.

As an example of the impact of comprehension and recall on defect resolution time, one study found that when engineers were tasked with resolving defects in code that had single-word identifiers vs code that had multi-word identifiers, they resolved issues 14% faster when given multi-word identifiers, which are more understandable than single-word identifiers [11]. They also "read the code more serially, jumping less frequently back and forth." [11] Another study found that "when finding semantic defects in

code with words instead of abbreviations and letters, participants found on average 19% more defects per minute and thus were faster when words were used as identifier names.” [12]

2.2. Productivity and happiness

The effect of naming on engineering efficiency is easy to measure on short timescales, as was done in the above studies. The longer-term impact of naming is studied less often. However, naming influences how engineers think about and communicate about software, which can impact productivity and happiness over longer timescales.

This impact is immediately obvious to engineers who are new to a team or project, as they experience an unpleasant spike in time wasted simply trying to understand concepts that are often more understandable than their names suggest. A codebase with poor understandability can cause a newcomer to take much more time to become productive than if the codebase was more understandable. On top of these costs, the newcomer may develop a poor perception of the project and in the worst case, a poor perception of the team.

The negative effects of a poorly-named codebase aren’t limited to newcomers, though. As described earlier, 60%-80% of software engineering life cycle costs come from maintenance. Codebases include huge numbers of identifiers, and engineers cannot remember the meaning of every single variable in a large codebase. When an

engineer needs to understand code that they haven't touched recently, they must rely on either recall to remember the meaning of previously-seen identifiers or rely on comprehension. Poor names can significantly increase the cost of both of these tasks.

The total cost of a single poorly-named identifier can be huge when the identifier needs to be repeatedly comprehended by multiple engineers over a long period of time. Many codebases last multiple years and contain thousands to millions of names of variables, classes, functions, and other identifiers. When these names have poor understandability, the sum of the total costs of all of the identifiers is massive.

In addition to the understandability costs that occur for a single engineer in isolation, there are also understandability costs involved in communication. When one engineer describes a concept to other engineers, poor names can increase the explanation time and create costly long-term confusion.

This not only makes engineering teams less inefficient but also impacts their happiness. Engineers who have ongoing productivity costs tend to be less satisfied with their work and consequently tend to be less productive [13, 14]. Poor understandability thus can create a vicious circle in which poor productivity and lack of satisfaction feed off of each other.

2.3. Career growth

In addition to the long-term impact of naming on productivity and happiness, the practice of good naming can also foster leadership skills. This practice helps engineers understand their domain deeply, develop influence within their organization and other leadership skills, and increase their opportunities to focus on higher-value work.

To be successful at naming, an engineer must understand a domain and its terminology deeply. This understanding can cause an engineer to develop more influence within an engineering team, as the team will start to seek their advice about the domain. Domain expertise is also often desired for leadership roles, as leadership requires making decisions that are informed by domain knowledge.

Being successful at naming also reduces the time that an engineer spends on inefficient, low-value work. By choosing understandable names and engendering good naming practices among others, they spend less time explaining concepts and names to others and more time on higher-value work.

Naming has a significant impact on many facets of software engineering. Its importance seems obvious, so why is naming still executed poorly so often? We need to understand what makes naming hard.

3. Why is naming hard?

> *There are only two hard things in Computer Science: cache invalidation and naming things.*
>
> *— Phil Karlton*

We've all struggled to grasp for the right name, only to come up short. Phil Karlton's quote about naming being one of the two hard problems in computer science perfectly captures the surprising complexity of a seemingly simple task.

Naming is one of the few skills in software engineering that has been required for the entirety of the field's existence, and yet we're still lousy at it. Why is naming so hard?

3.1. Dynamic, subjective requirements

An identifier's name is the communication of a dynamic concept to a dynamic audience. First, the namer must have an understanding of what the concept is. Newly-introduced concepts are often poorly-defined, and their definition often evolves over time. It can be difficult to know how a concept may change in the next iteration. The concept and its name are also highly dependent on the domain, which can also change over time.

The audience is also dynamic. The namer can guess who the audience of the name will be, but the audience may change over time. Engineers with different backgrounds and different amounts of domain experience will likely have to understand the name. In some cases, names must be understood by increasingly large and varied audiences, like a data model name that must be understood by product managers, designers, customer support, and even end-users. Any change in the original concept's definition, in the domain, or in the audience can make even a good name become a bad one.

The way in which a name's correctness is judged is subjective. For the name to be judged as being good by others, it must satisfy many requirements (e.g., understandability, conciseness) that are determined by the audience. These requirements can vary from person to person. One programming language community may use completely different requirements than another language community. Even the requirements that are commonly agreed upon (e.g., understandability) can be judged differently by different people.

Naming's difficulty isn't unique to software engineering. There are entire fields devoted to understanding naming (e.g., brand naming, terminology planning). Even across the general public, there is very little agreement on how to name things. Studies have shown that "the probability of having two people apply the same name to an object is between 7% and 18%, depending on the object." [15]

3.2. Insufficient best practices and tooling

Because of this dynamism, subjectivity, and complexity, there aren't currently well-defined, industry-wide standards for identifier naming, which further increases its difficulty. Best practices about naming within software engineering are mentioned in passing in chapters within books, blog posts, and articles but there hasn't been industry-wide standardization around what the universal best practices should be. Many of these resources discuss rules that belong within general principles (e.g., consistency, understandability), but there isn't a shared vocabulary for these, which makes the communication of them less unified and less widely understood.

Due to both the lack of industry-wide best practices and the complexity of naming, there is also a lack of tooling for enforcing high-quality naming. Naming is a category of software quality, but unlike other quality categories, naming has virtually no support within software quality tooling. There are few tools and libraries that verify the quality of their identifier names within static analysis or provide engineers with tools for improving their naming within their development environments.

Automated solutions for other software quality categories (e.g., vulnerability scanning, maintainability analysis) have benefited from industry-wide convergence on best practices and from tooling development, but naming has been relatively underserved.

3.3. Short-term costs and long-term value

It is also difficult to prioritize good naming because its costs are mostly immediate and its value is mostly long-term. When an engineer needs to decide between the short-term optimization of choosing a name that may be suboptimal or the long-term optimization of spending slightly more time finding a better name, they often choose the short-term option.

To choose a good name, an engineer must have developed experience with naming, have some domain experience, and take at least a small amount of time to choose the name. These are all costs, and if the value of a good name seems nebulous, then the logical decision is to not invest in naming. However, the value of good names is not nebulous. It has been studied and repeatedly proven to have short-term and long-term benefits.

3.4. How can we make naming easier?

To make high-quality naming easier to prioritize and perform, we need to do two things: we need to decrease its cost and increase its demonstrated value. Its value has been discussed in Why is naming important?, and if you're taking the time to read this book, you probably already believe that there is at least some value in it.

We should thus focus on decreasing the cost of high-quality naming. As with any hard problem in software engineering, we can break it down to better understand

how to solve it. We’ll develop principles and processes that can be used to methodically determine what the best name is for a given concept. When we finish, we’ll have a clear understanding of what constitutes a good name and how to choose one efficiently.

4. Scope of this book

Before we consider solutions, let’s determine what is in scope and out of scope. We’ll focus specifically on the naming of identifiers within software engineering. Identifier naming is the problem of choosing a name for a concept that needs to be represented within a codebase as an identifier, like a class, variable, or function.

To ensure that we focus precisely on solving this problem, we will explicitly not include discussions of adjacent topics. We will generally not discuss language-specific conventions, like the formatting and casing of identifiers (e.g., camel case vs. snake case). Because these topics are numerous, it would be difficult to provide a comprehensive perspective on them. They’re also typically already well-established within each language’s community, so there isn’t a great deal of value in discussing them in depth here, as more thorough, language-specific documentation about them already exists. Some language-specific naming conventions can impact comprehension, though, so we will touch on them briefly.

We will also explicitly avoid all naming topics other than identifier naming. Engineering teams often encounter other types of naming (e.g., product naming, brand naming, team naming), and we will not discuss these topics. A problem that’s related to identifier naming is how to determine the name and scope of domains, contexts, modules, and other groupings of concepts within a codebase. For example, how

do we decide what classes belong within a module, and how do we name that module? This problem would greatly increase the scope of this book, and it has been discussed at length elsewhere (e.g., Domain-Driven Design [16]), so we'll omit it as well.

Now that we've defined the scope of the problem, how do we solve it? Similar to how we've solved other software engineering problems, we can develop a set of principles that describe the best practices of solutions. We can then develop tools for pragmatically applying those principles.

II. Principles

5. Overview

How do we know if a name is good? There are universal principles for good naming, which are summarized below. We'll discuss each of these in subsequent chapters.

5.1. List of principles

Understandability
A name should describe the concept it represents.

Conciseness
A name should use only the words necessary to communicate the concept it represents.

Consistency
Names should be used and formatted uniformly.

Distinguishability
A name should be visually and phonetically distinguishable from other names.

6. Understandability

> *What is above all needed is to let the meaning choose the word, and not the other way around. In prose, the worst thing you can do with words is to surrender to them.*
>
> *— George Orwell*

6.1. Overview

A name should help the reader understand the concept that it represents as efficiently as possible. An understandable name has high comprehension (it can be understood quickly) and high recall (it can be remembered easily). For it to have both of these qualities, it must accurately describe the concept that it represents.

For example, if a variable that represents a bicycle is named “b”, then it is virtually incomprehensible without additional context. It is also difficult to remember that “b” corresponds to a bicycle. If it has a vague name like “item” or an incorrect name like “scooter”, then its comprehension and its recall will still be very low.

For a name to be understandable, the name must accurately describe the concept using memorable terminology, or it will be difficult to comprehend and recall. If the reader is familiar with the concept, the name should

be familiar to them, and if the reader is unfamiliar with the concept, then the concept's name should teach them the correct terminology for the concept within the domain.

6.2. Rules

6.2.1. Describe the concept

For a name to be understandable to the reader, it must describe the concept that it represents. If it describes a bicycle, it should be named `bicycle` and not be irrelevant, vague, or inaccurate.

```
# Bad
foo, thing, scooter

# Good
bicycle
```

6.2.2. Use dictionary terms

Names should use common terms that can be found in dictionaries, as these terms have better comprehension and recall than invented terms with which the reader will not already be familiar.

```
# Bad
o, org, people_group

# Good
organization
```

6.2.3. Use problem-domain terms

The “problem domain” contains the information that defines a problem (e.g., requirements, goals, etc), while the “solution domain” contains the aspects of the problem’s solution(s) (e.g., the solution’s design and implementation). [16] Problem domains are more stable and more fundamental than solution domains, so problem domain terminology should be preferred over solution domain terminology.

Let’s say you’re writing logic to optimize the scheduling of group meetings for businesses. If you use a scheduling algorithm that takes “events” as inputs, should you use the name “event” or “meeting?” Since “meeting” is the term that’s used in the problem domain, “meeting” is more understandable to more people. It makes the intent clear more quickly than a term that isn’t used in the problem domain.

```
# Bad
schedule_events(events)

# Good
schedule_meetings(meetings)
```

Finding the best problem-domain term for a given concept is a large topic will multiple strategies. For more information, see Domain-specific names.

6.2.4. Use standard terms for domain-agnostic concepts

Some concepts within a codebase are domain-agnostic because they are universal concepts (e.g., time, physical distance) or domain-agnostic technical concepts (e.g., concepts related to networking, operating systems, or databases). Given the universality of these, standardized terminology for them typically exists, and this terminology should be used.

For example, when deleting a row in a relational database, “delete” and “row” should be used instead of non-standard terms like “remove” and “record”.

```
# Bad
Database.remove_record(table, primary_key)

# Good
Database.delete_row(table, primary_key)
```

Similarly, when sending a kill signal to a unix process, “kill” and “pid” should be used instead of “stop” and “process ID”.

```
# Bad
Process.stop(process_id, signal_name)

# Good
Process.kill(pid, signal_name)
```

6.2.5. Use correct pluralization

If an identifier corresponds to or operates on a single entity, its name should be singular, not plural.

```
# Bad
users = User.where(id=user_id)[0]

# Good
user = User.where(id=user_id)[0]
```

If an identifier corresponds to or operates on a collection, its name should be plural, not singular.

```
# Bad
user = User.where(state='valid')

# Good
users = User.where(state='valid')
```

```
# Bad
validate_user(users)

# Good
validate_users(users)
```

6.2.6. Use accurate parts of speech

Each concept corresponds to a part of speech (e.g., a noun, a verb, an adjective), and its name should be that part of speech. While there are some exceptions, identifier types should typically be named using the following parts of speech:

Identifier type	Parts of speech	Examples
Classes	Nouns or noun phrases	`User`, `PaymentMethod`
Variables	Nouns, noun phrases, or linking verb and subject complement	`name`, `birth_date`, `is_valid`
Methods	Verbs, verb phrases, or linking verb and subject complement	`validate`, `delete_all`, `is_valid`
Interfaces	Nouns, noun phrases, or adjectives	`Parser`, `DataInput`, `Runnable`

For example, if there's a method on a `User` class that validates a user record, it should be named "validate" instead of "validation".

```
# Bad
user.validation()

# Good
user.validate()
```

If a method returns a boolean, like whether a user is valid, it should be a linking verb (e.g., "is") and a subject complement (e.g., "valid") or (if the language community encourages it) an adjective with a question mark.

```
# Bad
user.validate()

# Good
user.is_valid(), user.valid?()
```

6.2.7. Include units in measurements

When naming identifiers that represent measurements (e.g., a distance, an amount of time), the units of the measurement should always be included. Most measurements are meaningless without units. While units can sometimes be assumed, the costs of omitting them usually outweigh the costs of including them.

```
# Bad
elapsed_duration

# Good
elapsed_duration_in_days
```

```
# Bad
remaining_distance

# Good
remaining_distance_in_meters
```

```
# Bad
temperature

# Good
temperature_in_celsius
```

There are some cases in which it is reasonable to omit units. For example, if time measurements are used very frequently within a codebase and are consistently in seconds, or if the currency values within a codebase will always only be a single currency (e.g., USD, EUR), then the units will be clear enough that they can be assumed and omitted.

6.2.8. Avoid unconventional single-letter names

Single-letter variable names are relatively common: they may account for 10-20% of all variable names. [17] Conventions around their usage vary significantly by language, and some conventions encouraging their use are so strong that breaking them can be counterproductive. Kernighan and Pike write that “Local variables used in conventional ways can have very short names. The use of i and j for loop indices, p and q for pointers, and s and t for strings is so frequent that there is little profit and perhaps some loss in longer names”. [4]

When is it appropriate to use a single-letter name? If there’s a pervasive convention within the language around a specific letter and its meaning (e.g., `i` and `j` as loop indices), it will be understandable to most readers. Otherwise, use a name with full words. Multiple studies have shown that these improve both comprehension and recall among all of the demographics of engineers studied. [18, 19]

```
# Bad
u = users[0]

# Good
user = users[0]
```

A secondary motivation for full words is that some single-letter names can also be mistaken for other characters; the lowercase letter “l” can be mistaken for the number “1”,

and the lowercase letter “o” may be mistaken for the number “0”.

```
# Bad

l = users.length
row_count = l + 1

# Good

user_count = users.length
row_count = user_count + 1
```

When a variable has a relatively small scope (e.g., if it is only used on a single line), its comprehension and recall won’t be as negatively impacted as in a larger scope, as the reader can easily see all of its context at once. However, a full-word name will typically still be more understandable in this case, unless it’s long enough to significantly decrease legibility.

If additional logic is added within a single-letter name’s scope, though, it will become harder and harder to see all of the name’s scope at once. Finding the usage of a particular name (e.g., `user`) throughout a codebase can also be significantly harder or impossible when single-character names (e.g., `u`) are used.

```
# Bad

for u in users:
  print(u.name)

# Good

for user in users:
  print(user.full_name)
```

6.2.9. Avoid abbreviations

Abbreviations, including truncations (e.g., “org” for “organization”) and acronyms (e.g., “AP” for “accounts payable”), are less likely to be understandable than the full name, so their usage should be kept to a minimum. Widely-known abbreviations, like “URL”, “DNS”, and “ID”, are exceptions, as they don’t suffer the same downsides as lesser-known abbreviations.

For example, if “AP” is used to abbreviate “accounts payable”, a reader who is unfamiliar with the “AP” abbreviation will not understand what it means. Because it is a two-letter name, it may also be difficult to search for it in the codebase or in documentation.

Abbreviations are sometimes accompanied by inconsistency, as both the abbreviation and the full name are used to refer to a single concept. See Avoid abbreviations for additional discussion.

In some cases, the full name for a concept is so long that frequent use of it can become cumbersome, and an abbreviation may be worthwhile. When considering this option, consider its effect on conciseness, understandability, and consistency.

```
# Bad
ap, acts_payable

# Good
accounts_payable
```

```
# Bad
org
```

```
# Good
organization
```

6.2.10. Avoid non-standard symbolic names

Some languages support the use of symbolic names (e.g., a Scala function or a Ruby method could be named “<<–>>”). Some symbolic names (e.g., “+”, “-”, “==”) are so universal that they are understandable to the vast majority of software engineers, but there are many that are not. Symbolic names that aren’t universally acceptable are not immediately comprehensible. It’s difficult to search for them in a codebase, difficult to search the web to discover their meaning, difficult to pronounce them, and their meaning is difficult to remember. Similar to single-letter names, their brevity may be attractive to some engineers, but their lack of understandability typically outweighs this benefit.

For example, if you’re naming a Ruby method that invokes a function on every element within a collection, it should be named `map` instead of having a symbolic name.

```
# Bad
def ->(&block)

# Good
def map(&block)
```

6.2.11. Avoid cleverness

It can be tempting to inject humor or cleverness into what may otherwise be fairly boring code. This may be enjoyable in the short term, but its negative impact on understandability will outweigh its brief pleasure.

For example, if you're writing a method that removes bullets from text, you may consider using a more clever name about stopping bullets, like `apply_kevlar(text)`. Anyone else who reads this name will have to pause to consider its meaning, and in the worse case, they won't understand its meaning at all. You're adding more cognitive load to them, both for their initial encounter with it and for future encounters.

Cleverness also often reduces consistency: if someone is trying to find all of the methods that remove something from text, they may search for "remove" and will not find a method named `apply_kevlar(text)`.

```
# Bad
apply_kevlar(text)

# Good
remove_bullets(text)
```

6.2.12. Avoid usage of temporary or irrelevant concepts

Similar to names that use cleverness, names that rely on temporary or irrelevant concepts are less understandable and less consistent.

For example, if you're naming a method that kills a large number of processes, you might decide to pay homage to your favorite Metallica album and name it `kill_em_all`. Despite your belief in Metallica's pervasiveness, many people will not understand this reference, be confused by the odd naming choice, and have trouble remembering the name. A name like `kill_processes` will be more understandable and more timeless.

```
# Bad
kill_em_all(processes)

# Good
kill_processes(processes)
```

6.2.13. Consider the audience's familiarity with the name

Understandability is dependent on the audience that is interacting with the name. If a music application measures how positive or negative a song feels, this quality could be named either "valence" or "positivity." These are roughly synonyms, but "valence" is an industry term while "positivity" is known both inside and outside of the industry.

If the audience interacting with the name will be almost exclusively people who are already familiar with the term "valence", then it would make sense to use that industry-specific term, as it carries slightly more meaning than "positivity."

If the audience was unfamiliar with the term “valence”, then every time “valence” was read by or discussed with someone new, that person would need to take the time to find out what it means and would need to be able to remember its definition. In some cases this can be beneficial: it can teach the audience more about the industry, and if it has a more meaningful definition than “positivity”, then that meaning may be beneficial.

There are tradeoffs between choosing a name that the audience will immediately understand and one that has educational value, and it’s important to be intentional about weighing the benefits described above when making a choice like this.

7. Conciseness

The more you say, the less people remember.

— François Fénelon

7.1. Overview

Long names waste space: they use unnecessary visual space within code, unnecessary cognitive space when being recalled, and unnecessary temporal space when being spoken. Studies have found that long identifier names "require more time to consider, process, and, thus understand" [20] and that recall decreases as the number of syllables increases due to overcrowding of short-term memory. [19]

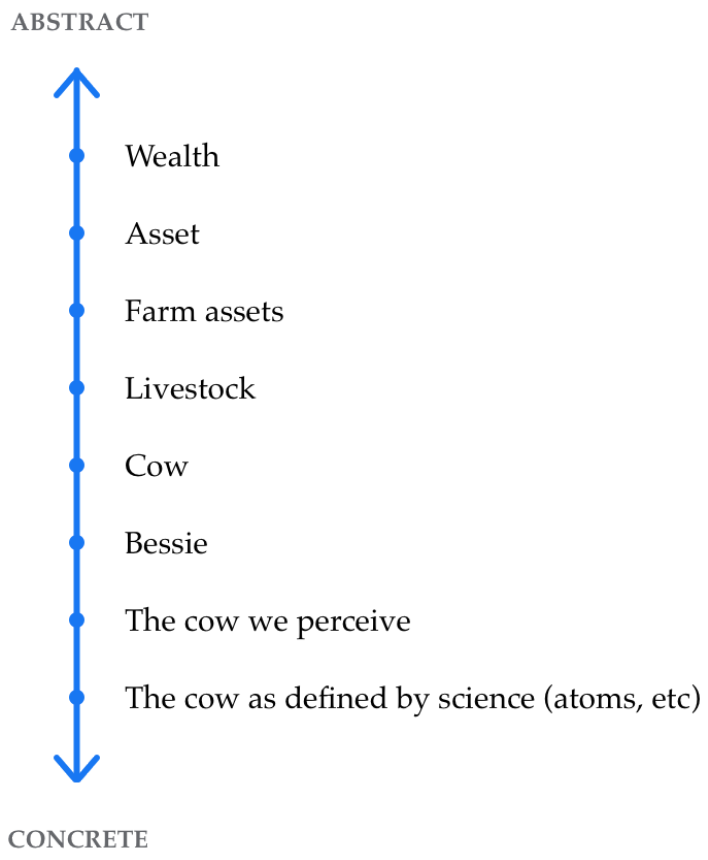
On the other hand, "code containing descriptive compound identifier names is comprehended faster than those with short single-word identifiers" [11], so names that are short but not descriptive decrease understandability. Indiscriminately shortening a name doesn't make it quicker to understand. The name's brevity and descriptiveness must be considered simultaneously: a good name communicates a large amount of information to the relevant audience with a small number of letters.

7.2. Rules

7.2.1. Use the appropriate level of abstraction

A name's level of abstraction indicates whether the name is specific, like `validated_total_sales_per_day_and_account_dataset`, or generic, like `data`, or somewhere in between. We need to decide what level of abstraction to use for a name.

In his 1939 book **Language in Thought and Action** [21], S. I. Hayakawato uses a "Ladder of Abstraction" to describe how people think and communicate in varying levels of abstraction. Hayakawato gives an example of a cow named "Bessie" and how it is represented at multiple levels of a ladder of abstraction:



Ladder of Abstraction

How should one refer to Bessie? The naming choice is dependent on the audience. The name “Bessie” may be the term that’s most relevant to Bessie’s owner, but “livestock” may be the term that’s most relevant to the owner of a large agriculture company.

Similarly, when introducing yourself, you might say one of the following phrases:

1. Hi, I’m a human

2. Hi, I’m Jane
3. Hi, I’m Jane Smith, daughter of Sarah Smith

If you said phrase 1. or phrase 3. to strangers at a casual party, you’d raise some eyebrows! However, phrase 1. may sound reasonable if you’re meeting an alien, and phrase 3. may sound reasonable if you’re meeting your mother’s colleagues in a formal setting.

Do not use a name that’s so specific that you’re providing information that’s irrelevant to the audience, and do not use a name that’s so generic that it provides little or no relevant information to them.

A good rule of thumb is to use a name that demonstrates the intent of the concept: what it is meant to be used for, and not *how* it is implemented. For example, let’s say that you need to write a method that removes leading and trailing whitespace when a phone number is displayed. Your new method looks like this:

```
class PhoneNumberPresenter:
  def my_new_method(phone_number):
    return phone_number.strip()
```

You could use one of the following names for your new method:

1. `process(phone_number)`
2. `format(phone_number)`
3. `trim_whitespace(phone_number)`
4. `strip(phone_number)`

The name `process` doesn't give the reader any relevant information about the operation being performed.

The name `format` tells the reader that the method will format the phone number, which is likely approximately the verb that they're expecting to find for a method that solves this requirement.

The name `trim_whitespace` tells the reader what the implementation is doing but doesn't provide information about its intended use.

The name `strip` tells the reader exactly what the implementation is without providing information about its intended use.

`format` is the name that best tells the reader the intent of the method: what it should be used for. This is the information that is relevant to the reader.

Details about its implementation should not be included in the name, as 1) they may change, 2) are not relevant for most readers, and 3) can always be seen by viewing the implementation.

Thus, a ladder of abstraction can be used to consider multiple names and then chose the name at the rung in the ladder that gives the reader the most relevant information about the concept without describing its implementation.

7.2.2. Use words with rich meaning

To maximize the amount of descriptiveness per letter, use words that communicate a great deal of relevant information.

For example, if you're designing the data schema of a music application, a collection of songs could be named `SongCollection`, but this is a relatively long name, and if that concept is equivalent to the concept of an "album", then "album" could convey more meaning with fewer letters.

```
# Bad
SongCollection

# Good
Album
```

Knowing the problem domain's terminology is important (see Use problem-domain terms), as the most popular term for a concept within a domain has often developed a fair amount of meaningfulness. Terms that are used frequently are also sometimes shorter than less-frequently used terms, as their brevity may have helped them become popular.

```
# Bad
ChildTask

# Good
Subtask
```

Method names can suffer from a lack of meaning due to the use of generic verbs. The verb "get" can easily be overused for method names in which a more specific verb would give the reader a better understanding of what the method does. For example, if a method makes a call to a third-party service to retrieve user information, consider naming it `fetch_user` instead of `get_user`.

```
# Bad
get_user(user_id)

# Good
fetch_user(user_id)
```

Some meaningful verbs that can be used to replace “get” include “find”, “fetch”, “generate”, “calculate”, and “load”. It’s important to have consistent usage of each of these terms and their semantics across a codebase.

7.2.3. Omit metadata

In many cases, the metadata about an identifier (e.g., its type, its intended scope) can be understood without reading its name, so including information about the type within the name is redundant and adds unnecessary length and noise. Identifier metadata should not be included in names.

```
# Bad
first_name_string, first_name_str

# Good
first_name
```

The inclusion of an unnecessary type suffix in the name of a collection is a more subtle variation:

```
# Bad
person_list, people_array

# Good
people
```

There are rare exceptions to this rule. For example, if a dynamic-typed language is used and knowledge of the type of a variable is crucial in a specific context, then including the type in the name may be worthwhile. Similarly, if two names represent a similar concept but have different types, a hint about the types within one or both of the names can make them more understandable.

```
# Bad

location = 'Chicago'
user.location = Location.geocode(location)

# Good

location_text = 'Chicago'
user.location = Location.geocode(location_text)
```

Interface names sometimes include an unnecessary suffix of `Interface`, even though the interface's usage typically makes it obvious that it is an interface. In the same way that a class would never be named `UserClass`, an interface should not an `Interface` suffix. To correct a violation like this, the interface's name can usually be converted into an adjective.

```
# Bad
ComparisonInterface

# Good
Comparable
```

7.2.4. Omit implementation details

The unnecessary inclusion of implementation details occurs most frequently within method names. A method's name should be based on *what* it does and not *how* it does it.

For example, say you're writing a method that processes the rows of a CSV in parallel. The fact that they're being processed in parallel is related to *how* the method is doing the processing, so it should be omitted in the name.

```
# Bad
csv_processor.process_in_parallel(rows)

# Good
csv_processor.process(rows)
```

If support for both serial and parallel processing is necessary, then the best convention would be to add an argument so that the method name always describes *what* it does, and the arguments describe *how* it does it.

```
# Bad

csv_processor.process_in_parallel(rows)
csv_processor.process_in_serial(rows)

# Good

csv_processor.process(rows, in_parallel=True)
csv_processor.process(rows, in_parallel=False)
```

Using arguments instead of defining multiple methods prevents the proliferation of methods when more configuration options are added and prevents an increase in the complexity of the names.

7.2.5. Omit unnecessary words

If a word does not significantly increase the reader's understanding of a concept, it should not be included in the name. For example, if a `User` class has a `delete_now` method, the word "now" is not necessary, as a reader would reasonably assume that a method named only `delete` would perform the deletion immediately. (If there is another method that deletes the user asynchronously, or if asynchronous deletion is common in the codebase, then there may be an argument for including it, though.)

```
# Bad
user.delete_now()

# Good
user.delete()
```

A variation of this is the redundant repetition of a class name within a method name, like a class method named `User.delete_user`. Because the class name already communicates the context, this method should just be named `delete`.

```
# Bad
User.delete_user(user)

# Good
User.delete(user)
```

Names of booleans are sometimes prefixed with "not" (e.g., `not_prepared`) or a negative prefix (e.g., `invalid`), which has two undesirable effects: it 1) increases the

length of the name without adding any meaning and 2) creates the opportunity for confusing double negatives within conditionals. Boolean names should always be stated in the positive to avoid these issues.

```
# Bad

if !user.is_invalid():
  user.save


# Good

if user.is_valid():
  user.save
```

8. Consistency

> *It is better to have a system…reflect one set of design ideas than to have one that contains many good but independent and uncoordinated ideas.*
>
> *— Fred Brooks, **The Mythical Man-Month** [22]*

8.1. Overview

Consistency encapsulates the uniformity and predictability of names. It can be broken down into three subprinciples:

- Each concept should only have one name
- Similar concepts should have similar names
- Similar names should have similar formats

Violations of any of these cause confusion and decrease comprehension, recall, searchability, and maintainability.

8.2. Rules

8.2.1. Obey popular naming conventions

Every popular language and framework has naming conventions that are widely accepted within their communities. Deviating from these conventions can have significant downsides.

Engineers familiar with the language will be surprised by a deviation and may struggle with the change or be reluctant to adapt to it. Third-party libraries and adjacent projects in the same language likely won't adhere to the deviation, so engineers will need to be able to shift between conventions, which can be mentally taxing.

One common category of deviations occurs when engineers familiar with one language work within an unfamiliar language and bring conventions from the familiar language to the new language. For example, a Java engineer may write Python using Java conventions, like camel-cased method names prefixed with `get` and `set` for getters and setters:

```
# Bad

class User:
  def __init__(self):
    self.name = None

  def self.get_name():
    return self.name

  def self.set_name(name):
    self.name = name


# Good

class User:
  def __init__(self, name):
    self.name = name
```

The best solution for preventing inadvertent deviations and enforcing intentional deviations is the use of a style

guide and a linter with well-developed, well-accepted rules. For more information on this, see Style guides.

8.2.2. Avoid synonyms

Multiple names should not be given to a single concept. For example, the concept of a “user” should not be given multiple names that have exactly the same meaning, like “user” and “end-user”.

If there is a single concept (e.g., the first name of a person) that is used in different contexts, it should always have the same name.

```
# Bad

class User:
  def __init__(self, full_name, first_name, last_name):
    self.full_name = full_name
    self.first_name = first_name
    self.last_name = last_name

class Person:
  def __init__(self, full_name, given_name, family_name):
    self.full_name = full_name
    self.given_name = given_name
    self.family_name = family_name

# Good

class User:
  def __init__(self, full_name, first_name, last_name):
    self.full_name = full_name
    self.first_name = first_name
    self.last_name = last_name

class Person:
  def __init__(self, full_name, first_name, last_name):
    self.full_name = full_name
    self.first_name = first_name
    self.last_name = last_name
```

Synonyms of more generic concepts should also be avoided. For example, if “start” and “initiate” are both used to refer to the same concept of starting a workflow, this creates an inconsistency which makes it difficult to perform a search for this concept and may confuse some readers. However, because the terms are understandable to a broad audience and have almost the same meaning, they’re less confusing than domain-specific synonyms, which are typically less understandable to first-time audiences.

```
# Bad

onboarding_workflow.start()
offboarding_workflow.initiate()

# Good

onboarding_workflow.start()
offboarding_workflow.start()
```

Having synonyms for a single concept creates confusion for everyone who interacts with those names. A new teammate may think that each name corresponds to a single, distinct concept. They may incorrectly assume that “end-user” represents a subset of a broader “user” concept. Similarly, anyone who interacts with the team in a temporary capacity may be confused about why “user” is being used in some places, while “end-user” is used elsewhere, even though they mean the same thing.

For more information on how to prevent and remove synonyms, see Style guides and Controlled vocabularies.

8.2.3. Avoid abbreviations

Abbreviations impact both understandability and consistency. See their discussion within Understandability.

8.2.4. Use similar names for similar concepts

Multiple sections of code are often conceptually similar to each other. For example, a model class may be similar to other model classes, or there may be multiple classes that all read a resource from an external service. The names given to these similar concepts and code should likewise be similar.

For example, if one method makes an HTTP request to an external service to retrieve a `Person` object, and another method makes an HTTP request to another external service to retrieve a `Company` object, the verbs used in these methods' names should be the same.

```
# Bad

# Both of these methods make an HTTP request
Person.fetch(person_id)
Company.get(company_id)

# Good

# Both of these methods make an HTTP request
Person.fetch(person_id)
Company.fetch(company_id)
```

The role that an identifier plays within a codebase also often has similarities with other identifiers. For example, if

a codebase has controller classes, then these should all be named using the same format.

```
# Bad

UserController
CompaniesController

# Good

UsersController
CompaniesController
```

8.2.5. Use consistent antonyms

Some concepts (e.g., “add”) have opposites (e.g., “remove”), and the names that are given to these opposites should be consistent. If the concepts are generic and not domain-specific, their names should be the most commonly-used names for those concepts.

All else being equal, antonyms with similar roots should be used:

```
# Bad

add/uninstall
install/remove

# Good

add/remove
install/uninstall
```

For examples of commonly-used antonyms, see Common antonyms.

9. Distinguishability

9.1. Overview

For a name to have high comprehension and recall, it should not have multiple meanings, and it should not be easily confused with another name. If it has multiple meanings, then an engineer will not always be confident about what it means and may even misinterpret it entirely both within code and when communicating about it. Similar-sounding and similar-looking names both have the same negative impact as names with multiple meanings: they create confusion and misuse, and they decrease comprehension, recall, and maintainability.

9.2. Rules

9.2.1. Avoid homographs and near-homographs

Homographs are two or more terms that have the same spelling but different meanings. The word “check” is an example, as it has multiple meanings:

- *(verb)* To examine, inspect, or look at appraisingly
- *(noun)* An examination or inspection of something
- *(noun)* A written and signed order for a bank to pay money

Homograph names cause confusion since the reader may not be able to quickly determine the name's meaning. For example, in an invoicing application, the word "check" could take on any of the meanings mentioned above, which can be confusing. Ideally, all usage of the word "check" would be replaced to eliminate any ambiguity, but practically, "check" may be the most understandable term for the concept of a written order for a bank to pay money.

```
# Bad

# Does this validate the invoice or return an instance of a
written check?
invoice.check()

# Good

# Use two distinctive names for the two distinct concepts
invoice.validate()
invoice.written_check()
```

Near-homographs occur when two or more terms have similar spellings, resulting in similar visual appearances. Two names that are long and have only a slight difference in spelling can be especially difficult to distinguish. Such names should be shortened and/or be modified so that they're more distinctive visually.

```
# Bad

class OrganizationOnboardingProcess:
class OrganizationOnboardingProcessor:

# Good

class Onboarding:
class OnboardingProcessor:
```

Differences in capitalization are not sufficient to make two names distinguishable. If the names represent different concepts, they should be changed to more precisely reflect their concepts.

```
# Bad

class Datastore:
class DataStore:

# Good

class SqlDatabase:
class InMemoryDatabase:
```

Names that differ only by one or two characters are also difficult to distinguish, especially when those characters look similar to each other, like the number `0` and the capital letter `O`. Single characters should not be used to abbreviate words within a name, but if they are, the characters should be easily distinguishable from one another.

```
# Bad

index_0 = 0
index_O = origin_point.index

# Good

initial_index = 0
origin_index = origin_point.index
```

For a list of particularly troublesome, similar-looking characters, see [Visually similar characters](#).

9.2.2. Avoid homophones and near-homophones

Homophones are two or more terms that have different spellings but the same pronunciation, like “whole” and “hole”. Homophones are often also homographs or near-homographs, but not always. Like homographs, homophones are easily confused with each other, but they’re confused when the names are spoken, rather than when the names are read.

Near-homophones are terms that have a similar pronunciation. As with homographs, they can be addressed by replacing them with more specific, more distinguishable terms.

```
# Bad

winnowed_results = winnow_results(results)
return window_results(winnowed_results)

# Good

valid_results = select_valid_results(results)
return window_results(valid_results)
```

Differentiation by capitalization is even more problematic for phonetic distinguishability than it is for visual distinguishability. Just as we saw with homographs, if the names represent different concepts, they should be changed to more precisely reflect their concepts.

```
# Bad

class Datastore:
class DataStore:
```

```
# Good

class SqlDatabase:
class InMemoryDatabase:
```

9.2.3. Avoid polysemes

A polyseme is a term that has multiple related definitions. For example, the word “book” may refer to any of the following concepts, which all exist within a single domain:

- A literary work (e.g., “Jane wrote a new book about software engineering”)
- A specific edition of a literary work (e.g., “Jane viewed the e-commerce website’s product page for her book”)
- A physical or digital instance of a literary work (e.g., “Jane took the book off the shelf”)

Because a polyseme’s definitions are often all relevant to a domain, it can be extremely difficult for a reader to confidently know what the term means. Even if the term always only corresponds to a single definition, new engineers will need to take time to learn that that’s the case and may inadvertently use the term incorrectly in the interim, creating confusing code. Even engineers who are experienced with the code may slip up and use the term incorrectly from time to time.

In the first snippet below, the word “book” refers to both an edition of a literary work (an ISBN is an identifier for editions of literary works) and to the physical copies of the

literary work. The second snippet corrects this by giving a new name (“physical copies”) to the second concept.

```
# Bad

book = Book.find_by_isbn(isbn)
book.books_in_stock_count

# Good

book = Book.find_by_isbn(isbn)
book.physical_copies_in_stock_count
```

9.2.4. Avoid names with distinct technical and non-technical meanings

This rule is an extension of Avoid homographs and near-homographs; it occurs often enough that it deserves its own discussion. The term “class” has both a technical definition (“a template for creating objects”) and non-technical definitions (“an educational course”, “a group of students that attend a course”, or “a group of students at the same level, like a ‘freshman class’”).

Avoid using these names when possible. “Class” is used so pervasively within engineering that a replacement name for its technical meaning isn’t realistic, but a replacement name for its domain meaning is feasible. If its domain meaning is “an educational course”, it could be replaced with “course” and thus remove any confusion about whether the name “class” is referring to the technical meaning or the domain meaning.

```
# Bad
```

```
class Class:

# Good

class Course:
```

Many other terms have distinct technical and non-technical meanings, like “event”, “message”, “window”, and “transaction”. If you’re working within a domain in which both meanings are relevant, be thoughtful about a decision to use such a term. In some cases, both the technical meaning and the domain meaning are pervasive enough that replacements for either aren’t feasible, but when replacements are feasible, use them.

III. Application

10. Overview

Principles provide general guidelines for how to name identifiers, but good naming also requires determining how to apply those principles and rules, as well as other considerations that are specific to identifier types, the domain, and the needs of the team and organization. We'll discuss these and provide practical solutions and tools for individuals and teams to apply these principles and cultivate good naming practices.

11. Tradeoffs

11.1. Overview

When principles conflict with each other, we need to decide which principle(s) to prioritize. To do this, we'll discuss common tradeoffs to consider when balancing multiple principles.

In general, consistency should be prioritized over other principles since inconsistency often violates all other principles. First, we'll describe the dominance of consistency over other principles, then we'll compare the three remaining principles with each other.

11.2. Consistency vs. other principles

Consistency not only provides the benefits that have already been discussed, but it also works to ensure that other principles are not violated. Inconsistency can:

- Violate **understandability** because there isn't a clear one-to-one mapping of names to concepts, preventing the reader from quickly understanding what each name means
- Violate **conciseness** because if there are two names for a single concept, then one of those names must not be the most concise name
- Violate **distinguishability** because it can result in a single name representing multiple concepts, or result

in two or more similar names representing the same concept

Engineers should thus be wary about prioritizing other principles over consistency, as inconsistency often has broader negative effects.

Additionally, if consistency exists, it's easier to address shortcomings in other principles. If a name is used consistently, but it violates another principle, it can be renamed.

11.3. Understandability vs. Conciseness

It can sometimes be unclear whether to prioritize understandability by using a longer, more specific name, or to prioritize conciseness by using a shorter, less specific name. Some examples include:

- `email` vs. `email_address`
- `name` vs. `full_name`
- `users_deactivated_in_last_10_days` vs. `users_deactivated_recently`

In some cases, distinguishability can be a tiebreaker. For example, let's say you're deciding whether to use `email` or `email_address` in a problem domain in which the word "email" can be used to mean either "an email message" or "an email address". `email` is more concise but less understandable. It is also less distinguishable since it could also be used to refer to a message. Thus, `email_address` is

preferable to `email`. A similar naming approach should be used when deciding between two names that have similar tradeoffs, like `name` vs. `full_name` or `phone` vs. `phone_number`.

However, the prevalence of the name should be considered as well. If the name is being used extremely frequently, then choosing a more concise name will reduce keystrokes and reduce the visual, cognitive, and audible space significantly. As an example, let's say there's a concept of "users deactivated in the last 10 days". If this concept only exists on one line in one code path and there are other user deactivation time windows (e.g., 30 days) used elsewhere in the codebase, it may be preferable to be specific and use a variable named something like `users_deactivated_in_last_10_days`.

However, if this concept exists in ten different code paths and is an important concept in the product domain, and there are no other user deactivation time windows, then using a long name can be disadvantageous. It takes many keystrokes to type and unnecessarily occupies a large amount of visual space and cognitive space. Because it's so prevalent, these disadvantages outweigh the advantages that specificity provides, and it's thus preferable to make it more concise by using a name like `users_deactivated_recently`.

11.4. Understandability vs. Distinguishability

Just because a term is the most commonly-used name for a concept in a given domain and thus appears to be the most understandable name, it may suffer from severe distinguishability shortcomings. In these cases, its indistinguishability should disqualify it, despite its apparent understandability. This often occurs with polysemes (e.g., “book”, “email”), which have different but related meanings.

“Book”, for example, may seem understandable, but it is a polyseme, as described in Avoid polysemes, and thus violates distinguishability. Instead of using a polyseme like “book”, a more distinguishable, concept-specific name should be used, like “literary work” or “physical copy”.

11.5. Conciseness vs. Distinguishability

Distinguishability also typically trumps conciseness. For example, “book” may seem more concise than and thus preferable to “literary work”. However, “book” is a polyseme and thus doesn’t clearly communicate the concept that it represents. As a result, it isn’t actually concise, because it isn’t actually communicating its meaning.

One shouldn’t try to increase a name’s conciseness so much that the name develops indistinguishability. If a name has multiple meanings, an engineer will not always be confident about what it means and may even misinterpret it

entirely. The cost of frequently being unsure about a name or misinterpreting it is very often higher than the cost of a distinguishable name being slightly longer than an indistinguishable name.

12. Identifier types

12.1. Overview

Naming conventions for each identifier type (e.g., classes, variables, methods, etc) vary across programming languages, development frameworks, and their communities, so we cannot provide many precise rules that strictly apply globally, but we will discuss some general principles. When using naming rules for identifier types, consistency of application is important for many of the same reasons that it's important in naming in general.

With identifier type-specific naming conventions, each language often has well-developed conventions (e.g., the casing of method names, or how to name methods that return booleans), and usage of those conventions makes a codebase understandable to engineers familiar with that language. So, consistency with language-specific conventions can be more important than strict adherence to all of the following recommendations.

12.2. Classes

Part(s) of speech: *Nouns or noun phrases (see Use accurate parts of speech)*

Since a class is a template for an object, its name should be a singular noun or a noun phrase that names that type of object.

The use of verbs as class names is discouraged. Robert C. Martin explicitly writes in Clean Code [1] that “a class name should not be a verb.” When a class has a verb as a name, it indicates that instances of it are actions, which is incorrect: the action occurs when a method is called.

```
# Bad

# This variable has an action as a name, but an action is not
taking place
validate_user = ValidateUser(user)

# Good

user_validator = UserValidator(user)
```

Additionally, when methods are called on the object (or on the class), they create a sequence of two verbs, but only the second verb is occurring, which can be confusing:

```
# Bad

# Only the email is being validated, even though the verb
"validate_user" appears multiple times
validate_user = ValidateUser(user)
validate_user.validate_email()

# Good

user_validator = UserValidator(user)
user_validator.validate_email()
```

If the casing convention is camel casing, use a single convention for camel-cased acronyms to avoid inconsistent casing. For example, when naming an XML parser class and an HTML parser class, they should be named either `XmlParser` and `HtmlParser`, or `XMLParser` and `HTMLParser`,

and not use inconsistent casing, like `XmlParser` and `HTMLParser`. This prevents a consistency violation.

12.3. Variables

Part(s) of speech: *Nouns, noun phrases, or linking verb/subject complement (see Use accurate parts of speech)*

Variables typically represent objects and values, so they should typically have names that are nouns or noun phrases. One exception to this is booleans, which should be named using a linking verb and subject complement, as described in Booleans. Instance variables, class variables, and fields are all similar enough in usage that these guidelines can be applied to them.

Variables should often use the same name as the relevant class or data type, unless there's a strong reason not to. In object-oriented programming, this means that a variable that represents an instance of a `User` should often have the word `user` in it:

```
# Bad
person = User.where(id=user_id)[0]

# Good
user = User.where(id=user_id)[0]
```

The exception to this is if there is a meaningful distinction between what the variable contains and the class that should be communicated. For example, in a blog application, it may be preferable to use `author` instead of

`user` when referring to a blog post's author, even if the class is `User`, especially if there are other `User` objects nearby or if the usage of the variable is only related to the user's role as an author.

If a variable is unused but necessary, like an unused variable used during multiple assignment, follow the programming language's convention for whether it should be unnamed. For example, if we're reading command-line arguments using Python's `sys.argv` and we don't use the first argument, it shouldn't be named:

```
# Bad

# If the `command` variable is unused but is still named, it adds
unnecessary screen space and cognitive load:
command, old_file_path, new_file_path = sys.argv

# Good

_, old_file_path, new_file_path = sys.argv
```

Since a variable's name should reflect the value that it represents, there are special guidelines for variables representing certain types of values, as described below.

12.3.1. Booleans

Boolean variables and boolean-returning methods correspond to a true or false value, and their names should explicitly indicate that. Their names should either be a linking verb and subject complement (e.g., `is_complete`, `has_admin_access`) or an adjective, especially if the language supports question marks at the end of methods

(e.g., `complete?`, `accessible_to_admins?`). This may vary across language conventions, though.

In particular, do not use names that could be misinterpreted as meaning a non-boolean concept. For example, if a variable stores a boolean representing whether or not there was an error, do not name it `error`. A reader is likely to interpret a variable named `error` as containing an exception object, rather than containing a boolean.

A common violation is the use of a noun that the boolean describes instead of the use of an adjective about the quality that the boolean describes. For example, if a method represents whether or not a process is complete, it should be named `is_complete` or `complete?`, not `status`. The word "status" indicates that there are multiple statuses, and a reader may assume that `status` represents a string or an enum.

```
# Bad

# What does this mean?!
if process.status():
  send_confirmation()

# Good

if process.is_complete():
  send_confirmation()
```

12.3.2. Collections

Collections should have plural names, as described in Use correct pluralization. They should also not include type

information in their name (e.g., suffixes of `_list` or `_array`), as described in Omit metadata.

12.3.3. Hash maps

Mappings of keys to values (e.g., hash maps, hash tables, dictionaries, etc) should have names that tell the reader exactly what they include. This is especially important in dynamically-typed languages where type inspection is difficult or impossible. Their names should include the names of their keys and values, like one of the following forms for a mapping of user IDs to user objects:

- `user_id_to_user`
- `user_ids_users`
- `users_by_id`

The first two forms above are slightly preferable to the last form since their words have the same order as the key and value, which is the order that readers typically think of for hash maps.

```
# Bad

# These two variable names don't make it clear what the variable
contains
user_dict = { 1: user_1, 2: user_2 }
user_ids = { 1: user_1, 2: user_2 }

# Good

# This variable name makes it clear that the variable contains a
mapping of user IDs to users
user_id_to_user = { 1: user_1, 2: user_2 }
```

```
# This name also makes iteration straightforward:
for user_id, user in user_id_to_user.items():
  print(user_id)
  print(user.full_name)
```

12.4. Methods

Part(s) of speech: *Verbs, verb phrases, or linking verb/subject complement (see Use accurate parts of speech)*

Method names should typically be verbs or verb phrases. One exception is methods that return booleans (see Booleans).

The name should describe what the method does. If a method does multiple things, the author should consider the single-responsibility principle to determine if there's a refactoring opportunity. If not, they should consider how to use the appropriate level of abstraction and omit implementation details.

Methods should ideally not have side effects, but if they do have a side effect, it should be mentioned in the name. A method named `find_user` should not also update the user. If it updates the user, it should have a name that indicates this.

Software applications often have a large quantity and variety of methods, and the details of these methods can change frequently. As a result, balancing conciseness and understandability is particularly important for method naming.

12.5. Method arguments

Part(s) of speech: *Nouns, noun phrases, or linking verb/subject complement (see [Use accurate parts of speech](#))*

While method arguments are similar to variables, some guidelines are specific to them. Method argument names typically indicate to the reader what data should be passed to the method, and they should make this very clear. Single-character names should be avoided since they make this particularly difficult:

```
# Bad

# It's unclear what type x and y should be, and in which
direction the copying is occurring
def copy_permissions(x, y):
  y.permissions = x.permissions
# It's unclear in which direction the copying is occurring
def copy_permissions(user_1, user_2):
  user_2.permissions = user_1.permissions

# Good

def copy_permissions(source_user, target_user):
  target_user.permissions = source_user.permissions
```

Similar to variables that are unused but necessary, method arguments that are unused but necessary should also follow the programming language's convention for whether they should be unnamed.

12.6. Interfaces

Part(s) of speech: *Nouns, noun phrases, or adjectives (see Use accurate parts of speech)*

Interface naming conventions vary by programming language, but the "Use the appropriate level of abstraction" rule and consistency are both particularly important when naming them. As an example, two common conventions are to use an "-able" suffix (e.g., `Writable`) or to use a "Can" prefix (e.g., `CanWrite`), but it's preferable to consistently use only a single convention when possible.

12.7. Constants

Part(s) of speech: *Nouns, noun phrases, or linking verb/subject complement (see Use accurate parts of speech)*

Each programming language typically has a strong convention on constant naming, which should be followed. The convention is often to use all uppercase with words separated by underscores (e.g., `MAX_FILE_SIZE_IN_BYTES`). If this is the convention, do not use more than one underscore between two words unless there is a clear convention for doing so.

12.8. Packages/modules/namespaces

Naming conventions for packages, modules, namespaces, and similar constructs vary by programming language and by organization. Depending on the context, these names

can either be literal (e.g., `configparsing`) or be figurative, like a project name (e.g., `zeus`). Package naming and project naming are large topics that are outside of the scope of this book, which focuses on identifier naming. Domain-Driven Design [16] is one starting point for these topics.

13. Style guides

A style guide is a set of conventions for how code should be written within a project, team, or organization. Style guides define enforceable rules around topics like file structure, code formatting, naming, and other conventions. Style guides' rules are typically automatically enforced with a linter that detects violations of the specified rules. A style guide accompanied by a linter is one of the best tools for enforcing consistency across one or more projects.

If you're new to style guides, a great place to start is to find a popular style guide for the language that you're using and apply it to your codebase. A style guide should be enforced automatically, ideally using a linter that runs within engineers' development environments, within code reviews, and within continuous integration pipelines.

Style guides often have rules about naming. Their naming rules often cover only a subset of the rules that we've discussed in this book, though, as it is difficult or impossible to automate many of these rules.

14. Controlled vocabularies

A controlled vocabulary is an organized collection of names and those names' definitions. Controlled vocabularies exist in many fields, and for good reason. They are used in information science, library science, and large organizations to mandate the use of specific names for specific concepts, as decided by their designers.

Controlled vocabularies help everyone in a field or an organization understand all of their domain's concepts and the appropriate names to use for them. Without a controlled vocabulary, names are created and applied in an ad hoc fashion, which inevitably generates inconsistency. Many software projects have terms that were hastily determined early on and persisted throughout the project's lifetime, causing a great deal of confusion.

The implementation of a controlled vocabulary depends on the size of the organization. In a small, five-person organization, it may be sufficient for one team member to create a single document: a glossary of their codebase's most commonly-used terms in a README, for example. The glossary may not be perfect, but given its low cost, it can yield a high ROI.

A larger organization will have more stakeholders in the terminology, so a more formal process for developing the controlled vocabulary and its ownership boundaries is required. A small, cross-functional team that includes at least one domain expert can draft an initial controlled

vocabulary, request feedback from interested parties, and ultimately formalize the vocabulary. The vocabulary can be stored in a doc, a wiki, or even in a controlled vocabulary management tool. Several open-source projects exist for this last option.

14.1. Further reading

Controlled vocabularies are a large field that is beyond the scope of this book. To learn more about the application of them within software engineering, you can read about domain-driven design [16], which refers to controlled vocabularies as “Ubiquitous Language.” To see examples of formal controlled vocabularies in a variety of contexts, see the ISO Standards catalog. The catalog is massive in scope and can be entertainingly specific. For example, ISO 3918:2007 defines “terms to be used in research work, official regulations, design, manufacture, installation and use of milking machines for cows, water buffaloes, sheep, goats or other mammals used for milk production” [23] and is roughly 35 pages long.

15. Renaming

We need to name concepts before we fully understand them or what their future meaning or audience will be. Given how many concepts engineers name, this can occur very frequently. Even with good naming practices, it's inevitable that ill-fitting names will be created. Once a name enters a codebase, it spreads throughout the codebase and becomes part of the team's vocabulary and the team's written output, including documents, slides, and knowledge bases.

Similar to any other tech debt, bad names cause ongoing costs that can compound over time. The natural solution is to perform a rename, which has a one-time cost that may be less than the cumulative ongoing costs of a bad name. When considering whether to perform a rename, be careful to consider the rename's scope, principal, interest, and process.

15.1. Scope

Renames can either be complete (renaming every instance of the old name) or partial (renaming instances of the old name only in specific components or systems). If you're considering a partial rename, consider its costs. The benefits of consistency listed above may be lost. This loss is more pronounced when the sections of code are used by the same engineers and less pronounced when the sections

of code are rarely used by the same engineers. In cases like this, consider a complete rename, so that the benefits of the new name are realized everywhere, and so that the costs of inconsistency aren't incurred.

In large systems, another option is to duplicate the old code, then perform the renames in the new code, gradually switch from the old code to the new code, and then finally delete the old code. This can reduce the risk and cost of any bugs caused by the rename.

In very large systems, consider how much scope the consistency should have. For example, in an email marketing application, it may seem worthwhile to start using the name "email address" instead of "email" to refer to an email address since the name "email" could be confusing, as it has other meanings in this domain. If the scope of the "email address" name included the entire application, then an `email` property on a `User` class should be renamed to `email_address`. However, the `User.email` property may not be used often in the same scope as, say, `Recipient.email_address` and `Campaign.from_email_address`, so the benefits of renaming it to `User.email_address` may not be worth the effort. Be intentional about determining the scope of a name.

15.2. Principal and interest

As with any technical debt, a bad name has interest and principal.

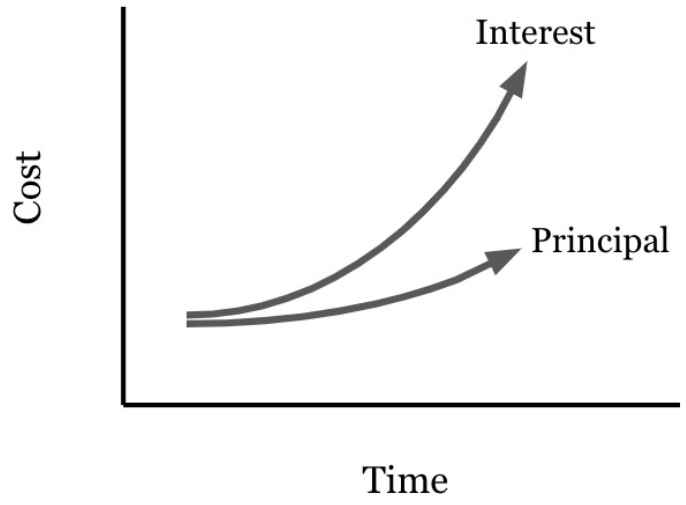
Interest

- **Definition**: The ongoing costs of a bad name
- **Costs**: Low comprehension and recall of concepts, high-effort communication, and the corresponding downstream effects (e.g., slower execution, lower morale)
- **Behavior over time**: These costs can compound over time, especially if the scope and audience of the name increase.

Principal

- **Definition**: The cost to pay down the debt of a bad name by renaming to a good name
- **Costs**: Renaming the name in code and docs, and communicating the rename
- **Behavior over time**: This cost can increase over time if the scope and audience of the name increase, but it may increase at a slower rate than the interest's rate of increase.

When deciding whether to pay off the debt of a bad name, compare the current principal to the expected future cumulative interest over the name's lifetime. If the cost of performing a rename today is lower than the future ongoing costs of the bad name over its lifetime, a rename is worthwhile.



The principal and interest of a bad name

15.2.1. Interest costs

Bad names cause the following ongoing costs:

- **Slow comprehension**: Existing team members take more time and more mental capacity to learn new parts of the codebase and their concepts, and new team members take more time and more mental capacity to ramp up on these parts of the codebase and their concepts.
- **Low recall**: After a team member learns a new name/concept, it takes them more time and more mental capacity to remember that name/concept in the future.
- **High-effort communication**: Communication about the name/concept within the team and with external

stakeholders takes more time and mental capacity since the names and concepts frequently need to be explained, disambiguated, or clarified.
- **Bugs**: Poor understanding of names and what they mean can easily lead to bugs.
- **Other downstream effects**: The additional time and mental capacity required by bad names can cause slower design and execution times. Confusion about concepts can lead to inadvertent defects or wasted effort due to miscommunication. All of these costs also result in opportunity costs: the team could be spending their time and mental capacity on higher-impact work. Finally, the net effect of these costs can lower morale, as the team is spending time on work that is unimportant and unfulfilling.

15.2.2. Principal costs

Performing a rename to pay down the debt has the following costs:

- **Name selection**: The new name needs to be determined.
- **Stakeholder alignment**: Before performing the rename, the stakeholders need to be aligned on the new name and its implementation plan.
- **Rename**: The bad name needs to be renamed to the good name in the codebase and optionally in the team's written output (e.g., docs, slides, etc). The rename may

interrupt other ongoing work on the same code paths, and depending on the nature of the rename, there's a small chance of introducing defects.
- **Bugs**: If a rename is performed incorrectly, it can cause bugs.
- **Switching costs**: Team members may need to spend some mental capacity to adjust to the new name and remember to use it instead of the old name in their code, docs, and communication.

15.3. Process

The process for a rename is dependent on the rename's scope and stakeholders, the nature of how the organization makes technical design decisions, and whether one concept or many concepts are being renamed. For non-trivial renames of a single concept, determine the following:

- **Scope**: Will the concept be renamed everywhere or only in specific components/systems?
- **Stakeholders**: Who has an interest in the rename? Who needs to review the rename, and who should be informed of the rename?
- **New name**: What is the proposed new name? This book's Principles can be used as a guide to help determine the appropriate new name.
- **Interest costs**: What are the ongoing costs of retaining the existing name?

- **Principal costs**: What are the costs of performing the rename?

Depending on the scope and stakeholders, either discuss and get alignment on the rename (for trivial/small renames) or write a document that describes the above topics (for non-trivial/large renames) and get approval on it from the relevant stakeholders. The inclusion of a list of historical costs can make the rename's value more compelling to stakeholders.

The exact process will depend on the nature of the organization. If you're not confident about whether a rename should be performed or the exact process for it, consider how other successful technical design decisions are made in your organization and follow a similar path.

16. Domain-specific names

The best name is typically the name that's used for the concept in the relevant problem domain, as described in Use problem-domain terms. How do you find this name, though, especially if you're unfamiliar with the domain and its terminology?

16.1. Consult domain-related resources

If you're trying to find what term is used most often for a concept within a domain, searching the web for the concept is a quick way to find out. Browse the websites of companies and products in the domain to see what name they use. In some industries, there's an organization that maintains standardized information or guidance for the industry, and their documentation often uses standard names.

Other potential resources include:

- API documentation - Typically describes a large number of concepts in a structured, technical format that can be explored quickly
- Product support forums and knowledge bases - Includes names that users are using to describe concepts
- Online communities (e.g., domain-specific news aggregators or subreddits) - Contain contemporary usage of names in the domain

- Wikipedia and academic papers

16.2. Consult domain experts

If you have access to experts within the domain, they are a great resource for determining the most relevant name. Colleagues in product, marketing, and content strategy often have a sense of the most understandable names within a product's domain, but colleagues in other functions can be very valuable, too, especially if they have been working in the domain for a long time.

16.3. Consult team members

You can also discuss the naming choice with colleagues, particularly ones who are working in the domain. A name that seems understandable to one person may be completely inscrutable to someone else, and it can be difficult to predict when this will be the case. Asking a colleague or two can take just a few minutes, and they may very well have an insight that makes the name much more understandable.

17. Developing naming skills

17.1. Improving your naming skills

The fact that you're reading this book shows that you're already ahead of the curve when it comes to thinking about naming, but our goal is to improve names by applying this information.

17.1.1. Initial steps

If you're a software engineer, you can start applying these principles today. An easy place to start is to give more consideration to small-scoped, new names in the code you work on. This involves both choosing better names when you need to name a new concept, and also giving thoughtful, useful feedback about new names that others introduce when you review their code. A slightly higher-effort but potentially more impactful step is to rename existing names that have a small scope, especially if you're already touching that code.

Start thinking more critically about existing names in the systems you work in to determine if any should be renamed, regardless of their scope. Discuss the existing names with your colleagues to see if those names are confusing or uninformative to others. This will also prompt your colleagues to start thinking more critically of those names, which helps to build early alignment on a rename.

If you want to be more ambitious, consider which names create the highest costs in your team, regardless of the rename cost. Try to plan the steps involved in replacing those. Is the benefit worth the cost? How would you communicate the benefits and costs to other team members?

17.1.2. Ongoing learning

Good naming requires domain knowledge and careful consideration about how to communicate concepts.

Learning your domain thoroughly can be a fast path to being informed about how to choose the best names within it. Find and read content that focuses on your domain to see what names typically map to each concept. This could include public knowledge bases (e.g., Wikipedia, forums), articles and blog posts by industry leaders in your domain, or any other relevant content. This helps you internalize your domain's effective controlled vocabulary and more effectively detect incorrect names.

Discuss names more frequently with people in other functions. Ask them if specific names are confusing or inconsistent with how non-engineers refer to that concept or ask them what they think about your team's names in general. Whenever it's unclear what the proper domain-specific name for a new concept is, ask users, product managers, marketing, or customer success-focused colleagues for their perspective to gain a wider perspective on the domain's terminology.

Pay attention to the vocabulary that both you and your colleagues use when discussing your domain in daily conversations. Are there terms that are part of the internal vernacular that are very obviously inaccurate or confusing?

Read and write more often. Well-written content, regardless of its format or domain, uses words that are precisely chosen. Reading it helps you observe how people who care a great deal about communication chose terms to communicate concepts. Writing forces you to select terms frequently. This practice fosters skills necessary for good naming.

17.2. Improving your team’s naming skills

Similar to other aspects of software engineering, naming is performed best when the team is aligned on the problem and actively collaborates on solving it. If you see a problem and are unsure if others see it, be proactive about raising it as a concern.

If anyone pushes back on the need for this discussion of names, ask them to think back to when they experience wasted effort or time due to poor naming. Be diligent but courteous about correcting anyone who uses the incorrect term for something. For example, if someone uses the term “Customer” when they should be using the term “User”, gently correct them. Otherwise, not only will that person continue to use the incorrect term, but everyone who heard them (including new team members) may use it as well.

When you join a new team, pay close attention to its names early on. If you are familiar with the domain, the names should be easy to understand. If you are unfamiliar with the domain, the names should teach you the appropriate terminology for the domain. In either case, the names should demonstrate the principles in this book. If that isn't the case, ask how the names were chosen. As a new team member, you may not have enough context to propose renames, but you have fresh eyes and may be able to more easily see poor names and gently ask if renaming has been considered or make note of them for later.

Regardless of which team you join, most engineers that you work with likely enjoy Phil Karlton's quote about naming being one of the two hard things in computer science. Good naming is hard, but it's also important. By using the practical application of sound principles, we can make it easier to find and use good names on any team, and thus make software engineering more efficient and enjoyable for everyone.

IV. Appendix

18. Common antonyms

For context, see Use consistent antonyms.

Term	Antonym(s)
acquire	release
add	remove, subtract
allocate	deallocate
allow	deny
assemble	disassemble
assign	deassign
attach	detach
begin	end
bind	unbind
commit	rollback
compile	decompile
compress	decompress
connect	disconnect
construct	destruct
create	destroy
enable	disable
encode	decode
encrypt	decrypt
enqueue	dequeue
enter	leave, exit
expand	collapse, contract
first	last
freeze	unfreeze
front	back
get	set
grant	revoke
head	tail
import	export

Term	Antonym(s)
include	exclude
increase	decrease
increment	decrement
indent	dedent
inflate	deflate
inject	eject
input	output
insert	delete
install	uninstall
link	unlink
load	unload
lock	unlock
maximum	minimum
new	old
next	previous
open	close
pause	resume
push	pop, pull
read	write
reference	dereference
register	deregister
select	deselect
send	receive
serialize	deserialize
set	get, unset, clear
set up	tear down
show	hide
source	destination, target

Term	Antonym(s)
start	stop
start up	shut down
up	down
upper	lower

19. Visually similar characters

For context, see Avoid homographs and near-homographs.

Character	Similar character(s)
`0` (zero)	`O` (capital o), `Q` (capital q), `D` (capital d)
`1` (one)	`I` (capital i), `l` (lowercase L)
`2` (two)	`Z` (capital z)
`4` (four)	`A` (capital a)
`5` (five)	`S` (capital s)
`6` (six)	`b` (lowercase B)
`7` (seven)	`T` (capital t)
`8` (eight)	`B` (capital b)
`9` (nine)	`g` (lowercase G)
`m` (lowercase M)	`rn` (lowercase R, lowercase N)
`n` (lowercase N)	`h` (lowercase H)

20. Rejected principles

The following principles were considered as candidates for the core naming principles, but they were rejected due to the reasons below.

20.1. Length

Length isn't a principle unto itself; the length of a name is dependent on Understandability and Conciseness.

20.2. Searchability

Names that are easily searchable ease the discovery of the usage of a name and its concept throughout codebases and knowledge bases. However, searchability isn't a valid principle, as it is dependent on Consistency and Distinguishability. If a name is consistently used and if it is distinguishable in spelling from other names, then it is searchable.

20.3. Pronounceability

A name that obeys Conciseness and Understandability will typically also be pronounceable. The exceptions to this are rare enough that pronounceability is more of a function of Conciseness and Understandability and not a principle unto itself.

20.4. Meaningfulness

“Meaningfulness” and “understandability” intersect each other in definition. However, “meaningfulness” has alternative definitions (e.g., “having a serious, important, or useful quality or purpose”) which blur its intent in the context of making a name understandable to its audience. “Understandability” is thus the preferred name for this principle.

20.5. Austerity

A name that obeys Conciseness and Understandability will typically also have neutrality, seriousness, and the other traits that austerity entails. “Austerity” is also not a terribly common word and can have different definitions for different people, so its understandability as a principle is not strong.

20.6. Accuracy

Accuracy is a subset of understandability: for a name to be understandable, it must be accurate.

20.7. Precision

In the context of identifier naming, precision may be interpreted as being equivalent to Use the appropriate level of abstraction. This is a subset of Conciseness.

20.8. Concision

“Concision” is a synonym for “conciseness.” “Conciseness” is more widely understood, though, so it is preferable to “concision.”

References

[1] Martin, R. C. (2008). Clean code: A handbook of agile software craftsmanship. In (1st ed.). USA: Prentice Hall PTR.

[2] McConnell, S. (2004). Code complete, second edition. In. USA: Microsoft Press.

[3] Hunt, A., & Thomas, D. (2000). *The pragmatic programmer: From journeyman to master*. USA: Addison-Wesley Longman Publishing Co., Inc.

[4] Kernighan, B. W., & Pike, R. (1999). *The practice of programming*. USA: Addison-Wesley Longman Publishing Co., Inc.

[5] Fowler, M. (1999). *Refactoring: Improving the design of existing code*. Boston, MA, USA: Addison-Wesley.

[6] Salviulo, F., & Scanniello, G. (2014). Dealing with identifiers and comments in source code comprehension and maintenance: Results from an ethnographically-informed study with students and professionals. In *Proceedings of the 18th international conference on evaluation and assessment in software engineering*. New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/2601248.2601251

[7] Feitelson, D. G., Mizrahi, A., Noy, N., Shabat, A. B., Eliyahu, O., & Sheffer, R. (2021). How developers choose

names. *CoRR*, *abs/2103.07487*. Retrieved from https://arxiv.org/abs/2103.07487

[8] Canfora, G., & Cimitile, A. (2001). Software maintenance. *Handbook of Software Engineering and Knowledge Engineering*, *1*. https://doi.org/10.1142/9789812389718_0005

[9] Pigoski, T., & April, A. (2004). Software maintenance. In.

[10] Deissenboeck, F., & Pizka, M. (2005). Concise and consistent naming. In *Software Quality Journal - SQJ* (Vol. 14, pp. 97–106). https://doi.org/10.1007/s11219-006-9219-1

[11] Schankin, A., Berger, A., Holt, D. V., Hofmeister, J. C., Riedel, T., & Beigl, M. (2018). Descriptive compound identifier names improve source code comprehension. In *Proceedings of the 26th conference on program comprehension* (pp. 31–40). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3196321.3196332

[12] Hofmeister, J. C., Siegmund, J., & Holt, D. V. (2017). Shorter identifier names take longer to comprehend. *Empirical Software Engineering*, *24*, 417–443.

[13] Graziotin, D., & Fagerholm, F. (2019). Happiness and the productivity of software engineers. In (pp. 109–124). https://doi.org/10.1007/978-1-4842-4221-6_10

[14] Graziotin, D., Fagerholm, F., Wang, X., & Abrahamsson, P. (2018). What happens when software developers are (un)happy. *Journal of Systems and*

Software, *140*, 32–47. https://doi.org/10.1016/j.jss.2018.02.041

[15] Butler, G., Grogono, P., Shinghal, R., & Tjandra, I. A. (1995). Retrieving information from data flow diagrams. *Proceedings of 2nd Working Conference on Reverse Engineering*, 22–29.

[16] Evans, E. (2003). *Domain-driven design: Tacking complexity in the heart of software*. USA: Addison-Wesley Longman Publishing Co., Inc.

[17] Beniamini, G., Gingichashvili, S., Orbach, A. K., & Feitelson, D. G. (2017). Meaningful identifier names: The case of single-letter variables. In *Proceedings of the 25th international conference on program comprehension* (pp. 45–54). Buenos Aires, Argentina: IEEE Press. https://doi.org/10.1109/ICPC.2017.18

[18] Lawrie, D., Morrell, C., Feild, H., & Binkley, D. (2006). What's in a name? A study of identifiers. In *IEEE International Conference on Program Comprehension* (Vol. 2006, pp. 3–12). https://doi.org/10.1109/ICPC.2006.51

[19] Lawrie, D. J., Morrell, C., Feild, H., & Binkley, D. W. (2007). Effective identifier names for comprehension and memory. *Innovations in Systems and Software Engineering*, *3*, 303–318.

[20] Binkley, D., Lawrie, D., Maex, S., & Morrell, C. (2009). Identifier length and limited programmer memory. *Science of Computer Programming*, *74*(7), 430–445. https://doi.org/https://doi.org/10.1016/j.scico.2009.02.006

[21] Hayakawa, S. I., & Hayakawa, A. R. (1990). *Language in thought and action*. Harcourt Brace Jovanovich. Retrieved from https://books.google.com/books?id=0H1p2sMdyXEC

[22] Brooks, F. P. (1995). *The mythical man-month (anniversary ed.)*. USA: Addison-Wesley Longman Publishing Co., Inc.

[23] International Organization for Standardization*Milking machine installations - Vocabulary* (Standard). (2007) (Vol. 2007). Geneva, CH.